In many cases, we want to employ algorithms that can progress from being model-free to planning with a model. For example, a robot learning how to walk may start to learn by trial and error (model-free), but once it has figured out the basics of walking, it can start to infer a model of its environment and then plan a sequence of steps to get from point A to B (model-based). We'll continue to explore on-policy, off-policy, model-based, and model-free algorithms in the rest of the book. In the next chapter we'll look at an algorithm that will help us build a network that can approximate the policy function.

Summary

- A *state-space* is the set of all possible states that the environment can be in. Usually the states are encoded as tensors, so the state space may be a vector of type $\mathbb{R}^n$ or a matrix in $\mathbb{R}^{n \times m}$.
- An *action-space* is the set of all possible actions given a state; for example, the action space for the game chess would be the set of all legal moves given some state of the game.
- A *state-value* is the expected sum of discounted rewards for a state given we follow some policy. If a state has a high state-value, that means that starting from this state will likely lead to high rewards.
- An *action-value* is the expected rewards for taking an action in a particular state. It is the value of a state-action pair. If you know the action-values for all possible actions for a state, you can decide to take the action with the highest action-value, and you would expect to receive the highest reward as a result.
- A *policy function* is a function that maps states to actions. It is the function that "decides" which actions to take given some input state.
- *Q function* is a function that takes a state-action pair and returns the action-value.
- *Q-learning* is a form of reinforcement learning where we attempt to model the Q function; in other words, we attempt to learn how to predict the expected rewards for each action given a state.
- A *deep Q-network (DQN)* is simply where we use a deep learning algorithm as the model in Q-learning.